

第 4 章 函数和递归

学习目标

- ☑ 掌握多参数、单返回值的数学函数的定义和使用方法
- ☑ 学会用 `typedef` 定义结构体
- ☑ 理解函数调用时用实参给形参赋值的过程
- ☑ 学会定义局部变量和全局变量
- ☑ 理解调用栈和栈帧，学会用 `gdb` 查看调用栈并选择栈帧
- ☑ 理解地址和指针
- ☑ 理解递归定义和递归函数
- ☑ 理解可执行文件中的正文段、数据段和 BSS 段
- ☑ 熟悉堆栈段，了解栈溢出的常见原因

运用前 3 章的知识尽管在理论上已经足以写出所有算法程序了，但实际上稍微复杂一点的程序往往由多个函数组成。函数是“过程程序设计”的自然产物，但也产生了局部变量、参数传递方式、递归等诸多新的知识点。本章的主要目的在于理解这纷繁复杂的、最后的语法。同时，通过 `gdb`，可以从根本上帮助读者理解，看清事物的本质。最后，通过一些实际的竞赛题目帮助读者学习编写算法程序的一般方法和技巧。

4.1 自定义函数和结构体

我们已经用过了许多数学函数，如 `cos`、`sqrt` 等。能不能自己写一个呢？没问题。下面就编写一个计算两点欧几里德距离的函数：

```
double dist(double x1, double y1, double x2, double y2)
{
    return sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
}
```

提示 4-1：C 语言中的数学函数可以定义成“返回类型 函数名(参数列表) { 函数体 }”，其中函数体的最后一条语句应该是“`return 表达式;`”。

这里，参数和返回值的类型一般是前面介绍过的“一等公民”，如 `int` 或者 `double`，也可以是 `char`。可不可以是数组呢？也不是不可以，但是比较麻烦，稍后再考虑。有时，函数并不需要返回任何值，例如，它只是用 `printf` 向屏幕输出一些内容。这时只需定义函数返回类型为 `void`，并且无须使用 `return`（除非希望在函数运行中退出函数）。

提示 4-2：函数的参数和返回值最好是“一等公民”，如 `int`、`char` 或者 `double` 等。其他“非一等公民”作为参数和返回值要复杂一些。如果函数不需要返回值，则返回类型应写成 `void`。

注意这里的 `return` 是一个动作，而不是描述。

提示 4-3：如果在执行函数的过程中碰到了 `return` 语句，将直接退出这个函数，不去执行后面的语句。相反，如果在执行过程中始终没有 `return` 语句，则会返回一个不确定的值。幸好，`-Wall` 可以捕捉到这一可疑情况并产生警告。

顺便说一句，`main` 函数也是有返回值的！到目前为止，我们总是让它返回 0，这个 0 是什么意思呢？尽管没有专门说明，读者应该已经发现了，`main` 函数是整个程序的入口。换句话说，有一个“其他的程序”来调用这个 `main` 函数——如操作系统、IDE、调试器，甚至自动评测系统。这个 0 代表“正常结束”，即返回给调用者。在算法竞赛中，除了有特殊规定之外，请总是让其返回 0，以免评测系统错误地认为程序异常退出了。

提示 4-4：在算法竞赛中，请总是让 `main` 函数返回 0。

函数不一定要一步得出结果。下面是上述函数的另一种写法：

```
double dist(double x1, double y1, double x2, double y2)
{
    double dx = x1-x2;
    double dy = y1-y2;
    return hypot(dx, dy);
}
```

这里用到了一个新的数学函数——`hypot`，相信读者能猜到它的意思^①。这个例子也说明，一个函数也可以调用其他函数——在自定义函数中写代码和在 `main` 函数中写代码并没有什么区别，以前讲过的知识都适用。

下面来思考一个问题：这个函数是否好用？通常，`x1` 和 `y1` 在语义上属于一个整体 (`x1,y1`)，而 `x2` 和 `y2` 属于另一个整体 (`x2,y2`)，代表两个点的坐标。那么能否设计一个函数，其参数是明显的两个点，而不是 4 个 `double` 型的坐标值呢？

```
struct Point{ double x, y; };

double dist(struct Point a, struct Point b)
{
    return hypot(a.x-b.x, a.y-b.y);
}
```

这里出现了一个新内容。上述代码中定义了一个称为 `Point` 的结构体，包含两个域：`double` 型的 `x` 和 `y`。

提示 4-5：在 C 语言中，定义结构体的方法为“`struct 结构体名称{ 域定义 };`”，注意花括号的后面还有一个分号。

这样用起来有些不合习惯：所有用到 `Point` 的地方都得写一个 `struct`。有一个方法可以

^① 注意：这个函数不是 ANSI C 的。

避开这些 struct，让结构体用起来和 int、double 这样的“原生”类型更接近：

```
typedef struct{ double x, y; }Point;

double dist(Point a, Point b)
{
    return hypot(a.x-b.x, a.y-b.y);
}
```

代码中虽然没少几个字符，但是看上去清爽多了！

提示 4-6：为了方便，往往用“typedef struct { 域定义; } 类型名;”的方式定义一个新类型名。这样，就可以像原生数据类型一样使用这个自定义类型。

计算组合数。编写函数，参数是两个非负整数 n 和 m ，返回组合数 $C_n^m = \frac{n!}{m!(n-m)!}$ ，

其中 $m \leq n \leq 25$ 。例如， $n=25$ ， $m=12$ 时答案为 5200300。

【分析】

既然题目中的公式多次出现 $n!$ ，将其作为一个函数编写是比较合理的：

程序 4-1 组合数（有问题）

```
long long factorial(int n){
    long long m = 1;
    for(int i = 1; i <= n; i++){
        m *= i;
    }
    return m;
}

long long C(int n, int m)
{
    return factorial(n) / (factorial(m) * factorial(n-m));
}
```

由此可见，编写函数并不困难。写完之后的函数可以像 cos、sqrt 等库函数一样被调用。

“别忘了测试！”如果你这样说，请为自己鼓掌。还记得第 2 章那个“阶乘”之和的第一个程序吗？那个程序溢出了。那这个程序呢？很不幸： $n=21$ ， $m=1$ 的返回值竟然是 -1。手算不难得到： $n=21$ ， $m=1$ 的正确结果是 21，显然结果不符。

提示 4-7：即使最终答案在所选择的数据类型范围之内，计算的中间结果仍然可能溢出。

这个题目还说明：即使认为题目在“暗示”你使用某种语言特性，也应该深入分析，不能贸然行事。如何避免中间结果溢出？办法是进行“约分”。一个简单的方法是利用 $n!/m! = (m+1)(m+2) \cdots (n-1)n$ 。虽然不能完全避免中间结果溢出，但是对于题目给出的范围已经可以保证得到正确的结果了。代码如下：

程序 4-2 组合数

```
long long C(int n, int m) {
    if(m < n-m) m = n-m;
    long long ans = 1;
    for(int i = m+1; i <= n; i++) ans *= i;
    for(int i = 1; i <= n-m; i++) ans /= i;
    return ans;
}
```

上述代码还有一个小技巧：当 $m < n-m$ 时把 m 变成 $n-m$ 。请读者思考这样做的意图。另外，这个函数里笔者改变了参数 m 的值。这样做并不会影响到函数的调用者，具体原因会在 4.2 节详细讨论。

提示 4-8：对复杂的表达式进行化简有时不仅能减少计算量，还能减少甚至避免中间结果溢出。

素数判定。编写函数，参数是一个正整数 n ，如果它是素数，返回 1，否则返回 0。

【分析】

根据定义，被 1 和它自身整除的、大于 1 的整数称为素数。这种“判断一个事物是否具有某一性质”的函数还有一个学术名称——谓词（predicate），下面程序中将写一个谓词。

程序 4-3 素数判定（有问题）

```
//n=1 或者 n 太大时请勿调用
int is_prime(int n)
{
    for(int i = 2; i*i <= n; i++)
        if(n % i == 0) return 0;
    return 1;
}
```

注意这里用到了两个小技巧。一是只判断不超过 $\sqrt{x}$ 的整数 i （想一想，为什么）。二是及时退出：一旦发现 x 有一个大于 1 的因子，立刻返回 0（假），只有最后才返回 1（真）。函数名的选取是有章可循的，“is_prime”取自英文“is it a prime?”（它是素数吗？）。

提示 4-9：建议把谓词（用来判断某事物是否具有某种特性的函数）命名成“is_xxx”的形式，返回 int 值，非 0 表示真，0 表示假。

注意程序 4-2 中 is_prime 函数上方的注释：不要用在 $n=1$ 或者 n 太大时调用。这是为什么呢？ n 太小时不难解释： $n=1$ 会被错误地判断为素数（因为确实没有其他因子）。 n 太大时的理由则不明显： $i*i$ 可能会溢出！如果 n 是一个接近 int 的最大值的素数，则当循环到 $i=46340$ 时， $i*i=2147395600 < n$ ；但 $i=46341$ 时， $i*i=2147488281$ ，超过了 int 的最大值，溢出变成负数，仍然满足 $i*i < n$ 。若 n 不是太大，可能出现 101128442 溢出后等于 2147483280，终止循环；但如果 $n=2147483647$ ，循环将一直进行下去。

提示 4-10: 编写函数时, 应尽量保证该函数能对任何合法参数得到正确的结果。如若不然, 应在显著位置标明函数的缺陷, 以避免误用。

下面是改进之后的版本:

程序 4-4 素数判定 (2)

```
int is_prime(int n)
{
    if(n <= 1) return 0;
    int m = floor(sqrt(n) + 0.5);
    for(int i = 2; i <= m; i++)
        if(n % i == 0) return 0;
    return 1;
}
```

除了特判 $n \leq 1$ 的情况外, 程序中还使用了变量 m , 一方面避免了每次重复计算 $\text{sqrt}(n)$, 另一方面也通过四舍五入避免了浮点误差——正如前面所说, 如果 sqrt 将某个本应是整数的值变成了 $\text{xxx}.99999$, 也将被修正, 但若直接写 $m = \text{sqrt}(n)$, “.99999” 会被直接截掉。

为什么 is_prime 的参数不是 long long 型呢? 因为当 n 很大时, 上述函数并不能很快计算出结果。对此, 在竞赛篇会有更详细的讨论。

4.2 函数调用与参数传递

4.1 节介绍的数学函数的特点是: 做计算, 然后返回一个值。但有时要做的并不是“计算”——如交换两个变量; 而有时则需要返回两个甚至更多的值——如解一个二元一次方程组, 函数仍然能满足需求, 但是规则会更复杂。根据笔者的经验, 这部分知识没搞清楚的初学者很容易在实战时出错, 所以这里介绍一些原理性的知识, 虽然有些枯燥, 但能帮助读者更好地理解。

4.2.1 形参与实参

程序 4-5 用函数交换变量 (错误)

```
#include<stdio.h>
void swap(int a, int b)
{
    int t = a; a = b; b = t;
}

int main()
{
    int a = 3, b = 4;
```

```

swap(3, 4);
printf("%d %d\n", a, b);
return 0;
}

```

读者应当还记得，这就是三变量交换算法。下面测试一下这个函数是否好用。很不幸，输出是“3 4”，而不是“4 3”。事实上，*a* 和 *b* 并没有被交换。为什么会这样呢？为了解释这一问题，请回忆“赋值”这个重要概念的含义。“诡异”的赋值语句 *a* = *a* + 1 是这样解释的：分为两步，首先计算赋值符号右边的 *a* + 1，然后把它装入变量 *a*，覆盖原来的值。那函数调用的过程又是怎样的呢？

第 1 步，计算参数的值。在上面的例子中，因为 *a* = 3, *b* = 4，所以 *swap*(*a*, *b*) 等价于 *swap*(3, 4)。这里的 3 和 4 被称为实际参数（简称实参）。

第 2 步，把实参赋值给函数声明中的 *a* 和 *b*。注意，这里的 *a* 和 *b* 与调用时的 *a* 和 *b* 是完全不同的。前面已经说过，实参最后将算出具体的值，*swap* 函数知道调用它的参数是 3 和 4，却不知道是怎么算出来的。函数声明中的 *a* 和 *b* 称为形式参数（简称形参）。

稍等一下，这里有个问题！这样一来，程序里有两个变量 *a*，一个在 *main* 函数里定义，一个是 *swap* 的形参，二者不会混淆吗？不会。函数（包括 *main* 函数）的形参和在该函数里定义的变量都被称为该函数的局部变量（*local variable*）。不同函数的局部变量相互独立，即无法访问其他函数的局部变量。需要注意的是，局部变量的存储空间是临时分配的，函数执行完毕时，局部变量的空间将被释放，其中的值无法保留到下次使用。与此对应的是全局变量（*global variable*）：此变量在函数外声明，可以在任何时候，由任何函数访问。需要注意的是，应该谨慎使用全局变量。

提示 4-11：函数的形参和在函数内声明的变量都是该函数的局部变量。无法访问其他函数的局部变量。局部变量的存储空间是临时分配的，函数执行完毕时，局部变量的空间将被释放，其中的值无法保留到下次使用。在函数外声明的变量是全局变量，可以被任何函数使用。操作全局变量有风险，应谨慎使用。

这样一来，函数的调用过程就可以简单理解成计算实参的值，赋值给对应的形参，然后把“当前代码行”转移到函数的首部。换句话说，在 *swap* 函数刚开始执行时，局部变量 *a* = 3, *b* = 4，二者的值是在函数调用时，由实参复制而来。

那么执行完毕后，函数又做了些什么呢？把返回值返回给调用它的函数，然后再次修改“当前代码行”，恢复到调用它的地方继续执行。等一下！函数是如何知道该返回到哪里继续执行的呢？为了解释这一问题，下面需要暂时把讨论变得学术一些——不要紧张，很快就会结束。

4.2.2 调用栈

还记得在讲解 *for* 循环时，笔者是如何建议的吗？多演示程序执行的过程，把注意力集中在“当前代码行”的转移和变量值的变化。这个建议同样适用于对函数的学习，只是要

增加一项内容——调用栈（Call Stack）。

调用栈描述的是函数之间的调用关系。它由多个栈帧（Stack Frame）组成，每个栈帧对应着一个未运行完的函数。栈帧中保存了该函数的返回地址和局部变量，因而不仅能在执行完毕后找到正确的返回地址，还很自然地保证了不同函数间的局部变量互不相干——因为不同函数对应着不同的栈帧。

提示 4-12: C 语言用调用栈（Call Stack）来描述函数之间的调用关系。调用栈由栈帧（Stack Frame）组成，每个栈帧对应着一个未运行完的函数。在 gdb^①中可以用 backtrace（简称 bt）命令打印所有栈帧信息。若要用 p 命令打印一个非当前栈帧的局部变量，可以用 frame 命令选择另一个栈帧。

在继续学习之前，建议读者试着调试一下刚才几个程序，除了关心“当前代码行”和变量的变化之外，再看看调用栈的变化。强烈建议读者在执行完 swap 函数的主体但还没有返回 main 函数之前，先看一下 swap 和 main 函数所对应的栈帧中 a 和 b 的值。如果受条件限制，在阅读到这里时没有办法完成这个实验，下面给出了用 gdb 完成上述操作的命令和结果。

第 1 步：编译程序。

```
gcc swap.c -std=c99 -g
```

生成可执行程序 a.exe（在 Linux 下是 a.out）。编译选项 -g 告诉编译器生成调试信息。编译选项 -std=c99 告诉编译器按照 C99 标准编译代码。

第 2 步：运行 gdb。

```
gdb a.exe
```

这样，gdb 在运行时会自动装入刚才生成的可执行程序。

第 3 步：查看源码。

```
(gdb) l
1      #include<stdio.h>
2      void swap(int a, int b){
3          int t = a; a = b; b = t;
4      }
5
6      int main(){
7          int a = 3, b = 4;
8          swap(3, 4);
9          printf("%d %d\n", a, b);
10         return 0;
```

这里(gdb)是 gdb 的提示符，字母 l 是输入的命令，为 list（列出程序清单）的缩写。正如代码所示，swap 函数的最后一行是第 4 行，当执行到这一行时，swap 函数的主体已经结

^① gdb 是一个功能强大的源码级调试器，虽然是基于命令的文本界面，但运用熟练后非常方便。关于 gdb 更多的介绍请参见附录 A。

束，但函数还没有返回。

第4步：加断点并运行。

```
(gdb) b 4
Breakpoint 1 at 0x401308: file swap.c, line 4.
(gdb) r
Starting program: D:\a.exe

Breakpoint 1, swap (a=4, b=3) at swap.c:4
4      }
```

其中，b命令把断点设在了第4行，r命令运行程序，之后碰到了断点并停止。

第5步：查看调用栈。

```
(gdb) bt
#0 swap (a=4, b=3) at swap.c:4
#1 0x00401356 in main () at swap.c:8
(gdb) p a
$1 = 4
(gdb) p b
$2 = 3
(gdb) up
#1 0x00401356 in main () at swap.c:8
8      swap(3, 4);
(gdb) p a
$3 = 3
(gdb) p b
$4 = 4
```

这一步是关键。根据bt命令，调用栈中包含两个栈帧：#0和#1，其中0号是当前栈帧——swap函数，1号是其“上一个”栈帧——main函数。这里甚至能看到swap函数的返回地址0x00401356，尽管不明确其具体含义。

使用p命令可以打印变量值。首先查看当前栈帧中a和b的值，分别等于4和3——这正是用三变量法交换后的结果。接下来用up命令选择上一个栈帧，再次使用p命令查看a和b的值，这次却得到3和4，为main函数中的a和b。前面讲过，在函数调用时，a、b只起到了“计算实参”的作用。但实参被赋值到形参之后，main函数中的a和b也完成了它们的使命。swap函数甚至无法知道main函数中也有着和形参同名的a和b变量，当然也就无法对其进行修改。最后要用q命令退出gdb。

用了这么多篇幅解释调用栈和栈帧，是因为无数的经验告诉笔者：理解它们对于今后的学习和编程是至关重要的，特别是递归——初学者学习语言的最大障碍之一，调用栈将有助于理解。

4.2.3 用指针作参数

在了解了刚才的 swap 函数不能奏效的原因后，应该如何编写 swap 函数呢？答案是用指针。

程序 4-6 用函数交换变量（正确）

```
#include<stdio.h>
void swap(int* a, int* b)
{
    int t = *a; *a = *b; *b = t;
}

int main()
{
    int a = 3, b = 4;
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

怎么样，是不是觉得不太习惯，却又有点似曾相识呢？不太习惯的是 int 和 a 中间的乘号，而似曾相识的是 swap(&a, &b) 这种变量名前面加 “&” 的用法——到目前为止，唯一采取这种用法的是 scanf 系列函数，而只有它改变了实参的值！

变量名前面加 “&” 得到的是该变量的地址。什么是“地址”呢？

提示 4-13：C 语言的变量都是放在内存中的，而内存中的每个字节都有一个称为地址（address）的编号。每个变量都占有一定数目的字节（可用 sizeof 运算符获得），其中第一个字节的地址称为变量的地址。

下面用 gdb 来调试上面的程序，看看它和程序 4-5 有什么不同。前 4 步是一样的，可直接看调用栈。

```
(gdb) bt
#0 swap (a=0x22ff74, b=0x22ff70) at swap2.c:4
#1 0x0040135c in main() at swap2.c:8
(gdb) p a
$1 = (int *) 0x22ff74
(gdb) p b
$2 = (int *) 0x22ff70
(gdb) p *a
$3 = 4
(gdb) p *b
```

```
$4 = 3
(gdb) up
#1 0x0040135c in main() at swap2.c:8
8      swap(&a, &b);
(gdb) p a
$5 = 4
(gdb) p b
$6 = 3
(gdb) p &a
$7 = (int *) 0x22ff74
(gdb) p &b
$8 = (int *) 0x22ff70
```

在打印 `a` 和 `b` 的值时，得到了诡异的结果——`(int *) 0x22ff74` 和 `(int *) 0x22ff70`。数值 `0x22ff74` 和 `0x22ff70` 是两个地址（以 `0x` 开头的整数以十六进制表示，在这里暂时不需了解细节），而前面的 `(int *)` 表明 `a` 和 `b` 是指向 `int` 类型的指针。

提示 4-14：用 `int* a` 声明的变量 `a` 是指向 `int` 型变量的指针。赋值 `a = &b` 的含义是把变量 `b` 的地址存放在指针 `a` 中，表达式 `*a` 代表 `a` 指向的变量，既可以放在赋值符号的左边（左值），也可以放在右边（右值）。

注意：`*a` 是指“`a` 指向的变量”，而不仅是“`a` 指向的变量所拥有的值”。理解这一点相当重要。例如，`*a = *a + 1` 就是让 `a` 指向的变量自增 1。甚至可以把它写成 `(*a)++`。注意不要写成 `*a++`，因为“`++`”运算符的优先级高于“取内容”运算符“`*`”，实际上会被解释成 `*(a++)`。

有了指针，C 语言变得复杂了很多。一方面，需要了解更多底层的内容才能彻底解释一些问题，包括运行时的地址空间布局，以及操作系统的内存管理方式等。另一方面，指针的存在，使得 C 语言中变量的说明变得异常复杂——你能轻易地说出用 `char * const (*next)()` 声明的 `next` 是什么类型的吗^①？毫不夸张地说，指针是程序员（不仅是初学者）杀手。

既然如此，那应当如何使用指针呢？别忘了本书的背景——算法竞赛。算法竞赛的核心是算法，没有必要纠缠如此复杂的语言特性。了解底层的细节是有益的（事实上，前面已经介绍了一些底层细节），但在编程时应尽量避免，只遵守一些注意事项即可。

提示 4-15：千万不要滥用指针，这不仅会把自己搞糊涂，还会让程序产生各种奇怪的错误。事实上，本书的程序会很少使用指针。

再次回到对正确 `swap` 程序的调试。在 `swap` 程序中，`a` 和 `b` 都是局部变量，在函数执行完毕以后就不复存在了，但是 `a` 和 `b` 里保存的地址却依然有效——它们是 `main` 函数中的局部变量 `a` 和 `b` 的地址。在 `main` 函数执行完毕之前，这两个地址将始终有效，并且分别指向 `main` 函数的局部变量 `a` 和 `b`。程序交换的是 `*a` 和 `*b`，也就是 `main` 函数中的局部变量 `a` 和 `b`。

^① 这是一个指向函数的指针，该函数返回一个指针，该指针指向一个只读的指针，此指针指向一个字符变量。

4.2.4 初学者易犯的错误

这个 swap 函数看似简单，但初学者还是很容易写错。一种典型的错误写法是：

```
void swap(int* a, int* b)
{
    int *t = a; a = b; b = t;
}
```

此写法交换了 swap 函数的局部变量 a 和 b(辅助变量 t 必须是指针。int t = a 是错误的)，但却始终没有修改它们指向的内容，因此 main 函数中的 a 和 b 不会改变。另一种错误写法是：

```
void swap(int* a, int* b)
{
    int *t;
    *t = *a; *a = *b; *b = *t;
}
```

这个程序错在哪里？t 是一个指向 int 型的指针，因此 *t 是一个整数。用一个整数作为辅助变量去交换两个整数有何不妥？事实上，如果用这个函数去替换程序 4-6，很可能会得到“43”的正确结果。为什么笔者要坚持说它是错误的呢？

问题在于，t 存储的地址是什么？也就是说 t 指向哪里？因为 t 是一个变量（指针也是一个变量，只不过类型是“指针”），所以根据规则，它在赋值之前是不确定的。如果这个“不确定的值”所代表的内存单元恰好是能写入的，那么这段程序将正常工作；但如果它是只读的，程序可能会崩溃。读者可尝试赋初值 int *t = 0，看看内存地址“0”能不能写。

至此，终于初步理解了地址和指针。尽管只是初步理解，但是为将来的学习奠定了良好的基础。指针有很多巧妙但又令人困惑的用法。如果有一种语法，但在完整地学习了本书后始终没有看到此语法被使用，那么这通常意味着这个语法不必学（至少在算法竞赛中不必用到）。事实上，笔者在编写本书的例程时，首先考虑的是要通俗易懂，避开复杂的语言特性，其次才是简洁和效率。

4.2.5 数组作为参数和返回值

如何把数组作为参数传递给函数？先来看下面的例子。

程序 4-7 计算数组的元素和（错误）

```
int sum(int a[]) {
    int ans = 0;
    for(int i = 0; i < sizeof(a); i++)
        ans += a[i];
    return ans;
}
```

这个函数是错误的，因为 `sizeof(a)` 无法得到数组的大小。为什么会这样？因为把数组作为参数传递给函数时，实际上只有数组的首地址作为指针传递给了函数。换句话说，在函数定义中的 `int a[]` 等价于 `int *a`。在只有地址信息的情况下，是无法知道数组里有多少个元素的。

正确的做法是加一个参数，即数组的元素个数。

程序 4-8 计算数组的元素和（正确）

```
int sum(int* a, int n) {
    int ans = 0;
    for(int i = 0; i < n; i++)
        ans += a[i];
    return ans;
}
```

在上面的代码中，直接把参数 `a` 写成了 `int* a`，暗示 `a` 实际上是一个地址。在函数调用时 `a` 不一定非要传递一个数组，例如：

```
int main() {
    int a[] = {1, 2, 3, 4};
    printf("%d\n", sum(a+1, 3));
    return 0;
}
```

提示 4-16：以数组为参数调用函数时，实际上只有数组首地址传递给了函数，需要另加一个参数表示元素个数。除了把数组首地址本身作为实参外，还可以利用指针加减法把其他元素的首地址传递给函数。

指针 `a+1` 指向 `a[1]`，即 2 这个元素（数组元素从 0 开始编号）。因此函数 `sum` “看到” `{2, 3, 4}` 这个数组，因此返回 9。一般地，若 `p` 是指针，`k` 是正整数，则 `p+k` 就是指针 `p` 后面第 `k` 个元素，`p-k` 是 `p` 前面的第 `k` 个元素，而如果 `p1` 和 `p2` 是类型相同的指针，则 `p2-p1` 是从 `p1` 到 `p2` 的元素个数（不含 `p2`）。下面是 `sum` 函数的另外两种写法。

程序 4-9 计算左闭右开区间内的元素和（两种写法）

写法一：

```
int sum(int* begin, int* end) {
    int n = end - begin;
    int ans = 0;
    for(int i = 0; i < n; i++)
        ans += begin[i];
    return ans;
}
```

写法二：

```
int sum(int* begin, int* end) {
    int *p = begin;
```

```

int ans = 0;
for(int *p = begin; p != end; p++)
    ans += *p;
return ans;
}

```

其中写法一先进行了一次指针减法，算出了从 `begin` 到 `end`（不含 `end`）的元素个数 `n`，然后再像前面那样把 `begin` 作为“数组名”进行累加。写法二看起来更“高级”，事实上也更具一般性，用一个新指针 `p` 作为循环变量，同时累加其指向的值。这两个函数的调用方式与之前相似，例如，声明了一个长度为 10 的数组 `a`，则它的元素之和就是 `sum(a, a+10)`；若要计算 `a[i], a[i+1], ..., a[j]`，则需要调用 `sum(a+i, a+j+1)`。

`sum` 的最后两种写法及其调用方式非常重要（将在第 5 章中继续讨论），请读者仔细体会。

把数组作为指针传递给函数时，数组内容是可以修改的。因此如果要写一个“返回数组”的函数，可以加一个数组参数，然后在函数内修改这个数组的内容。不过在算法竞赛中经常采取其他做法，原因在第 5 章会做进一步的说明。

4.2.6 把函数作为函数的参数

把函数作为函数的参数？看上去挺奇怪的，但实际上有一个非常典型的应用——排序。

例题 4-1 古老的密码（Ancient Cipher, NEERC 2004, UVa1339）

给定两个长度相同且不超过 100 的字符串，判断是否能把其中一个字符串的各个字母重排，然后对 26 个字母做一个一一映射，使得两个字符串相同。例如，JWPUDJSTVP 重排后可以得到 WJDUPSJPVT，然后把每个字母映射到它前一个字母（B→A, C→B, ..., Z→Y, A→Z），得到 VICTORIOUS。输入两个字符串，输出 YES 或者 NO。

【分析】

既然字母可以重排，则每个字母的位置并不重要，重要的是每个字母出现的次数。这样可以先统计出两个字符串中各个字母出现的次数，得到两个数组 `cnt1[26]` 和 `cnt2[26]`。下一步需要一点想象力：只要两个数组排序之后的结果相同，输入的两个串就可以通过重排和一一映射变得相同。这样，问题的核心就是排序。

C 语言的 `stdlib.h` 中有一个叫 `qsort` 的库函数，实现了著名的快速排序算法。它的声明是这样的：

```

void qsort ( void *base, size_t num, size_t size, int (* comparator) ( const
void *, const void * ) );

```

前 3 个参数不难理解，分别是待排序的数组起始地址、元素个数和每个元素的大小。最后一个参数比较特别，是一个指向函数的指针，该函数应当具有这样的形式：

```

int cmp(const void *, const void *) { ... }

```

这里的新内容是指向常数的“万能”的指针：`const void *`，它可以通过强制类型转化变成任意类型的指针。对于本题来说，排序的对象是整型数组，因此要这样写：

```
int cmp ( const void *a , const void *b ) {
    return *(int *)a - *(int *)b;
}
```

一般地，需要先把参数 a 和 b 转化为真实的类型，然后让 `cmp` 函数当 $a < b$ 、 $a = b$ 和 $a > b$ 时分别返回负数、0 和正数即可。学会排序之后，本题的主程序并不难编写，读者不妨一试。

是不是觉得上面那个 `cmp` 看起来非常别扭？的确如此。虽然 `qsort` 是 C 语言的标准库函数，但在算法竞赛中一般不使用它，而是使用 C++ 中的 `sort` 函数。此函数将在第 5 章中介绍。本节的主要目的是告诉读者，“将一个函数作为参数传递给另外一个函数”是很有用的。

4.3 递归

终于到了本书 C 语言部分的最后一站——递归了。很多人都认为递归是语言中最难理解的内容之一，但也不要紧张：如果认真理解了 4.2 节中的指针、地址和调用栈，会发现递归其实是一个很自然的东西。

4.3.1 递归定义

递归的定义如下：

递归：

参见“递归”。

什么？这个定义什么也没有说啊！好吧，改一下：

递归：

如果还是没明白递归是什么意思，参见“递归”。

噢，也许这次你明白了，原来递归就是“自己用到自己”的意思。这个定义显然比上一个要好些，因为当你终于悟出其中的道理后，就不必继续“参见”下去了。事实上，递归的含义比这要广泛。

A 经理：“这事不归我管，去找 B 经理。”于是你去找 B 经理。

B 经理：“这事不归我管，去找 A 经理。”于是你又回到了 A 经理这儿。

接下来发生的事情就不难想到了。只要两个经理的说辞不变，你又始终听话，你将会永远往返于两个经理之间。这叫做无限递归（Infinite Recursion）。尽管在这里，A 经理并没有让你找他自己，但还是回到了他这里。换句话说，“间接地用到自己”也算递归。

回忆一下，正整数是如何定义的？正整数是 1, 2, 3, …… 这些数。这样的定义也许对于小学生来说是没有任何问题的，但当你开始觉得这个定义“不太严密”时，你或许会喜欢这样的定义：

(1) 1 是正整数。

(2) 如果 n 是正整数， $n+1$ 也是正整数。

(3) 只有通过(1)、(2)定义出来的才是正整数^①。

这样的定义也是递归的：在“正整数”还没有定义完时，就用到了“正整数”的定义。这和前面的“参见递归”在本质上是相同的，只是没有它那么直接和明显。

同样地，可以递归定义“常量表达式”（以下简称表达式）：

(1) 整数和浮点数都是表达式。

(2) 如果 A 是表达式，则 (A) 是表达式。

(3) 如果 A 和 B 都是表达式，则 A+B、A-B、A*B、A/B 都是表达式。

(4) 只有通过(1)、(2)、(3)定义出来的才是表达式。

简洁而严密，这就是递归定义的优点。

4.3.2 递归函数

数学函数也可以递归定义。例如，阶乘函数 $f(n)=n!$ 可以定义为：

$$\begin{cases} f(0)=1 \\ f(n)=f(n-1)\times n \quad (n\geq 1) \end{cases}$$

对应的程序如下：

程序 4-10 用递归法计算阶乘

```
#include<stdio.h>
int f(int n)
{
    return n == 0 ? 1 : f(n-1)*n;
}
int main()
{
    printf("%d\n", f(3));
    return 0;
}
```

提示 4-17：C 语言支持递归，即函数可以直接或间接地调用自己。但要注意为递归函数编写终止条件，否则将产生无限递归。

4.3.3 C 语言对递归的支持

尽管从概念上可以理解阶乘的递归定义，但在 C 语言中函数为什么真的可以“自己调用自己”呢？下面再次借助 gdb 来调试这段程序。

首先用 **b f** 命令设置断点——除了可以按行号设置外，也可以直接给出函数名，断点将设置在函数的开头。下面用 **r** 命令运行程序，并在断点处停下来。接下来用 **s** 命令单步执行：

^① 更严密的说法是：正整数集是满足(1)、(2)的最小集。这里牺牲一点严密性，换来的是更通俗易懂的表达方式。

```
(gdb) r
Starting program: C:\a.exe

Breakpoint 1, f (n=3) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=2) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=1) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=0) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s
4      }
```

看到了吗？在第一次断点处， $n=3$ （3 是 main 函数中的调用参数），接下来将调用 $f(3-1)$ ，即 $f(2)$ ，因此单步一次后显示 $n=2$ 。由于 $n=0$ 仍然不成立，继续递归调用，直到 $n=0$ 。这时不再递归调用了，执行一次 s 命令以后会到达函数的结束位置。

接下来该做什么？没错！好好看看下面的调用栈吧！

```
(gdb) bt
#0 f (n=0) at factorial.c:4
#1 0x00401308 in f (n=1) at factorial.c:3
#2 0x00401308 in f (n=2) at factorial.c:3
#3 0x00401308 in f (n=3) at factorial.c:3
#4 0x00401359 in main () at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=1) at factorial.c:4
#1 0x00401308 in f (n=2) at factorial.c:3
#2 0x00401308 in f (n=3) at factorial.c:3
#3 0x00401359 in main () at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=2) at factorial.c:4
#1 0x00401308 in f (n=3) at factorial.c:3
```



```
#2 0x00401359 in main() at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=3) at factorial.c:4
```

```
#1 0x00401359 in main() at factorial.c:6
```

```
(gdb) s
```

```
6
```

```
main() at factorial.c:7
```

```
7      return 0;
```

```
(gdb) bt
```

```
#0 main() at factorial.c:7
```

每次执行完 `s` 指令，都会有一层递归调用终止，直到返回 `main` 函数。事实上，如果在递归调用初期查看调用栈，则会发现每次递归调用都会多一个栈帧——和普通的函数调用并没有什么不同。确实如此。由于使用了调用栈，C 语言自然支持了递归。在 C 语言的函数中，调用自己和调用其他函数并没有任何本质区别，都是建立新栈帧，传递参数并修改当前代码行。在函数体执行完毕后删除栈帧，处理返回值并修改当前代码行。

提示 4-18：由于使用了调用栈，C 语言支持递归。在 C 语言中，调用自己和调用其他函数并没有本质不同。

如果仍然无法理解上面的调用栈，可以作如下的比喻。

皇帝（拥有 `main` 函数的栈帧）：大臣，你给我算一下 `f(3)`。

大臣（拥有 `f(3)` 的栈帧）：知府，你给我算一下 `f(2)`。

知府（拥有 `f(2)` 的栈帧）：县令，你给我算一下 `f(1)`。

县令（拥有 `f(1)` 的栈帧）：师爷，你给我算一下 `f(0)`。

师爷（拥有 `f(0)` 的栈帧）：回老爷，`f(0)=1`。

县令：（心算 $f(1)=f(0)*1=1$ ）回知府大人，`f(1)=1`。

知府：（心算 $f(2)=f(1)*2=2$ ）回大人，`f(2)=2`。

大臣：（心算 $f(3)=f(2)*3=6$ ）回皇上，`f(3)=6`。

皇帝满意了。

虽然比喻不甚恰当，但也可以说明一些问题。递归调用时新建了一个栈帧，并且跳转到了函数开头处执行，就好比皇帝找大臣、大臣找知府这样的过程。尽管同一时刻可以有多个栈帧（皇帝、大臣、知府同时处于“等待下级回话”的状态），但“当前代码行”只有一个。

读者如果理解了这个比喻，但仍不理解调用栈，不必强求，知道递归为什么能正常工作即可。设计递归程序的重点在于给下级安排工作。

4.3.4 段错误与栈溢出

至此，对 C 语言的介绍已近尾声。别忘了，我们还没有测试 `f` 函数。也许你会说：不

必了, 我知道乘法会溢出——算阶乘时, 乘法老是会溢出。可这次不一样了。把 `main` 函数的 `f(3)` 换成 `f(100000000)` 试试 (别数了, 有 8 个 0)。什么? 没有输出? 不对呀, 即使溢出, 也应该是个负数或者其他 “显然不对” 的值, 不应该没有输出啊!

`gdb` 再次帮了我们的忙。用 `-g` 编译后用 `gdb` 载入, 二话不说就用 `r` 执行。结果发现 `gdb` 报错了!

```
(gdb) r
Starting program: C:\a.exe

Program received signal SIGSEGV, Segmentation fault.
0x00401303 in f (n=99869708) at 4-6.c:3
3      return n == 0 ? 1 : f(n-1)*n;
```

`gdb` 中显示程序收到了 `SIGSEGV` 信号——段错误。这太让人沮丧了! 眼看本章就要结束了, 怎么又遇到一个段错误? 别急, 让我们慢慢分析。我保证, 这是本章最后的难点。

你有没有想过, 编译后产生的可执行文件里都保存着些什么内容? 答案是和操作系统相关。例如, `UNIX/Linux` 用的 `ELF` 格式, `DOS` 下用的是 `COFF` 格式, 而 `Windows` 用的是 `PE` 文件格式 (由 `COFF` 扩充而来)。这些格式不尽相同, 但都有一个共同的概念——段。

“段” (`segmentation`) 是指二进制文件内的区域, 所有某种特定类型信息被保存在里面。可以用 `size` 程序^①得到可执行文件中各个段的大小。如刚才的 `factorial.c`, 编译出 `a.exe` 以后执行 `size` 的结果是:

```
D:\>size a.exe
      text    data     bss     dec     hex filename
    2756      740      224     3720     e88 a.exe
```

此结果表示 `a.exe` 由正文段、数据段和 `bss` 段组成, 总大小是 3720, 用十六进制表示为 `e88`。这些段是什么意思呢?

提示 4-19: 在可执行文件中, 正文段 (`Text Segment`) 用于储存指令, 数据段 (`Data Segment`) 用于储存已初始化的全局变量, `BSS` 段 (`BSS Segment`) 用于储存未赋值的全局变量所需的空

是不是少了点什么? 调用栈在哪里? 它并不储存在可执行文件中, 而是在运行时创建。调用栈所在的段称为堆栈段 (`Stack Segment`)。和其他段一样, 堆栈段也有自己的大小, 不能被越界访问, 否则就会出现段错误 (`Segmentation Fault`)。

这样, 前面的错误就不难理解了: 每次递归调用都需要往调用栈里增加一个栈帧, 久而久之就越界了。这种情况叫做栈溢出 (`Stack Overflow`)。

提示 4-20: 在运行时, 程序会动态创建一个堆栈段, 里面存放着调用栈, 因此保存着函数的调用关系和局部变量。

^① `Linux` 和 `Windows` 下的 `MinGW` 中都有这个程序。

那么栈空间究竟有多大呢？这和操作系统相关。在 Linux 中，栈大小是由系统命令 `ulimit` 指定的，例如，`ulimit -a` 显示当前栈大小，而 `ulimit -s 32768` 将把栈大小指定为 32MB。但在 Windows 中，栈大小是储存在可执行文件中的。使用 `gcc` 可以这样指定可执行文件的栈大小：`gcc -Wl,--stack=16777216①`，这样栈大小就变为 16MB。

提示 4-21：在 Linux 中，栈大小并没有储存在可执行程序中，只能用 `ulimit` 命令修改；在 Windows 中，栈大小储存在可执行程序中，用 `gcc` 编译时可以通过 `-Wl,--stack=<byte count>` 指定。

聪明的读者，现在你能理解为什么在介绍数组时，建议“把较大的数组放在 `main` 函数外”了吗？别忘了，局部变量也是放在堆栈段的。栈溢出不一定是递归调用太多，也可能是局部变量太大。只要总大小超过了允许的范围，就会产生栈溢出。

4.4 竞赛题目选讲

从技术上讲，不用函数和递归也可以写出所有程序^②。但是从实用的角度来讲，函数和递归能帮我们大忙。人毕竟不是机器，代码的可读性和可维护性是相当重要的。很多初学者渴望学习到更好的调试技巧，但在此之前，笔者却总是建议他们先学习如何更好地写程序。如果方法得当，不仅能更快地写出更短的程序，而且调试起来也更轻松，隐含的错误也会更少。本节的题目并不涉及新的知识点，但在程序组织和调试技巧上会给读者一些新的启示。

例题 4-2 刽子手游戏 (Hangman Judge, UVa 489)

刽子手游戏其实是一款猜单词游戏，如图 4-1 所示。游戏规则是这样的：计算机想一个单词让你猜，你每次可以猜一个字母。如果单词里有那个字母，所有该字母会显示出来；如果没有那个字母，则计算机会在一幅“刽子手”画上填一笔。这幅画一共需要 7 笔就能完成，因此你最多只能错 6 次。注意，猜一个已经猜过的字母也算错。

在本题中，你的任务是编写一个“裁判”程序，输入单词和玩家的猜测，判断玩家赢了 (You win.)、输了 (You lose.) 还是放弃了 (You chickened out.)。每组数据包含 3 行，第 1 行是游戏编号 (-1 为输入结束标记)，第 2 行是计算机想的单词，第 3 行是玩家的猜测。后两行保证只含小写字母。

样例输入：

```
1
cheese
```

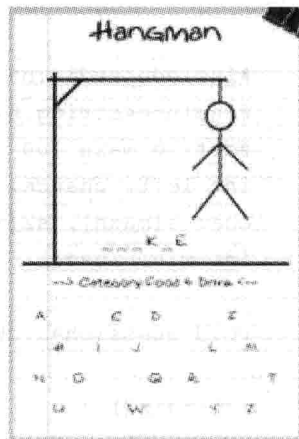


图 4-1 刽子手游戏

^① 实际上，栈大小是由连接程序 `ld` 指定的。`gcc` 编译参数 `-Wl` 的作用正是把其后的参数 (`--stack=<size>`) 传递给 `ld`。

^② 这里没有“几乎”二字。函数和递归均可以用其他内容替代。

```

cheese
2
cheese
abcdefg
3
cheese
abcdefghijkl
-1

```

样例输出：

```

Round 1
You win.
Round 2
You chickened out.
Round 3
You lose.

```

【分析】

一般而言，程序不是直接从第一行开始写到最后一行结束，而是遵循两种常见的顺序之一：自顶向下和自底向上。什么叫自顶向下呢？简单地说，就是先写框架，再写细节。实际上，之前已经用过这个方法了，就是先写“伪代码”，然后转化成实际的代码。有了“函数”这个工具之后，可以更好地贯彻这个方法：先写主程序，包括对函数的调用，再实现函数本身。自底向上和这个顺序相反，是先写函数，再写主程序。对于编写复杂软件来说，自底向下的构建方式有它独特的优势^①。但在算法竞赛中，这样做的选手并不多见^②。

程序 4-11 刽子手游戏——程序框架

```

#include<stdio.h>
#include<string.h>
#define maxn 100
int left, chance;           //还需要猜 left 个位置，错 chance 次之后就会输
char s[maxn], s2[maxn];    //答案是字符串 s，玩家猜的字母序列是 s2
int win, lose;              //win=1 表示已经赢了；lose=1 表示已经输了

void guess(char ch) { ... }

int main() {
    int rnd;
    while(scanf("%d%s", &rnd, s, s2) == 3 && rnd != -1) {
        printf("Round %d\n", rnd);
        win = lose = 0;
        //求解一组新数据之前要初始化
    }
}

```

^① 有兴趣的读者可以翻阅 Paul Graham 的经典著作《On Lisp》。

^② 注意：这里讨论的是编写代码的顺序。在测试时，先测试工具函数的方式非常常用。

```

left = strlen(s);
chance = 7;
for(int i = 0; i < strlen(s2); i++) {
    guess(s2[i]);           //猜一个字母
    if(win || lose) break;  //检查状态
}
//根据结果进行输出
if(win) printf("You win.\n");
else if(lose) printf("You lose.\n");
else printf("You chickened out.\n");
}
return 0;
}

```

有一些细节需要说明。

一是变量名的选取。那个 `rnd` 本应叫 `round`，但是有一个库函数也叫 `round`，所以改名叫 `rnd` 了。当然，改成 `Round` 也可以，因为 C 语言的标识符是区分大小写的。这里改成 `rnd` 只是个人习惯。毕竟这个代码很短，而且 `rnd` 这个变量的作用域很小，很容易搞清楚它的含义。在第 5 章学习完 STL 之后，这种“被用过的常用名字”还会增加，例如 `count`、`min`、`max` 等都是 STL 已经使用的名字，程序中最好避开它们。

二是变量的使用。全局变量本应该尽量少用，但是对于本题来说，需要维护的内容比较多，例如，是否赢了，是否输了，以及剩余的机会数等。如果不用全局变量，则它们都需要传递给函数 `guess`。更麻烦的是，其中有些参数还需要被 `guess` 修改，只能传指针，但这会让代码变“丑^①”。所以笔者最终选择了使用全局变量。读者完全可以对此持不同看法，刚才的文字只是想说明：变量和函数调用方式的设计是一个需要思考的问题。如果设计出的方案还未写出便觉得别扭，恐怕写出来的程序会既不优美，也不好调试，甚至容易隐藏 bug。

下一步是实现 `guess` 函数。在编写这个函数时，可能会注意到一个问题：题目中说了猜过的字母再猜一次算错，可是似乎并没有保存哪些字母已经猜过。一个解决方案是在程序框架中增加一个字符数组 `int guessed[256]`，让 `guessed[ch]` 标识字母 `ch` 是否已经猜过。但其实还有一个更简单的方法，就是将猜对的字符改成空格，像这样：

程序 4-12 刽子手游戏——`guess` 函数

```

void guess(char ch) {
    int bad = 1;
    for(int i = 0; i < strlen(s); i++)
        if(s[i] == ch) { left--; s[i] = ' '; bad = 0; }
    if(bad) --chance;
    if(!chance) lose = 1;
    if(!left) win = 1;
}

```

^① 当然，这是笔者的主观看法。有些人觉得充满指针的代码很优美。

这样,程序就完整了。如何调试呢?每猜完一个字母之后打印出 `s`、`left`、`chance` 等重要变量的值,很容易就能发现程序出错的位置,读者不妨一试。另一方面,如果刚才加上了 `guessed` 数组,每次打印的调试信息就会多出这样一个庞大的数组,不仅数据多,而且不直观,会给调试带来麻烦。一般来说,减少变量的个数对于编程和调试都会有帮助。

例题 4-3 救济金发放 (The Dole Queue, UVa 133)

$n(n < 20)$ 个人站成一圈,逆时针编号为 $1 \sim n$ 。有两个官员, A 从 1 开始逆时针数, B 从 n 开始顺时针数。在每一轮中,官员 A 数 k 个就停下来,官员 B 数 m 个就停下来(注意有可能两个官员停在同一个人上)。接下来被官员选中的人(1 个或者 2 个)离开队伍。

输入 n, k, m 输出每轮里被选中的人的编号(如果有两个人,先输出被 A 选中的)。例如, $n=10, k=4, m=3$, 输出为 4 8, 9 5, 3 1, 2 6, 10, 7。注意:输出的每个数应当恰好占 3 列。

【分析】

仍然采用自顶向下的方法编写程序。用一个大小为 0 的数组表示人站成的圈。为了避免人走之后移动数组元素,用 0 表示离开队伍的人,数数时跳过即可。主程序如下:

```
#include<stdio.h>
#define maxn 25
int n, k, m, a[maxn];

//逆时针走 t 步, 步长是 d (-1 表示顺时针走), 返回新位置
int go(int p, int d, int t) { ... }

int main() {
    while(scanf("%d%d%d", &n, &k, &m) == 3 && n) {
        for(int i = 1; i <= n; i++) a[i] = i;
        int left = n; //剩下的人数
        int p1 = n, p2 = 1;
        while(left) {
            p1 = go(p1, 1, k);
            p2 = go(p2, -1, m);
            printf("%3d", p1); left--;
            if(p2 != p1) { printf("%3d", p2); left--; }
            a[p1] = a[p2] = 0;
            if(left) printf(",");
        }
        printf("\n");
    }
    return 0;
}
```

注意 `go` 这个函数。当然也可以写两个函数:逆时针 `go` 和顺时针 `go`, 但是仔细思考后发现这两个函数可以合并:逆时针和顺时针数数的唯一区别只是下标是加 1 还是减 1。把这

个+1/-1抽象为“步长”参数，就可以把两个 go 统一了。代码如下：

```
int go(int p, int d, int t) {
    while(t-->0) {
        do { p = (p+d+n-1) % n + 1; } while(a[p] == 0); //走到下一个非0数字
    }
    return p;
}
```

例题 4-4 信息解码 (Message Decoding, ACM/ICPC World Finals 1991, UVa 213)

考虑下面的 01 串序列：

0, 00, 01, 10, 000, 001, 010, 011, 100, 101, 110, 0000, 0001, ..., 1101, 1110, 00000, ...

首先是长度为 1 的串，然后是长度为 2 的串，依此类推。如果看成二进制，相同长度的后一个串等于前一个串加 1。注意上述序列中不存在全为 1 的串。

你的任务是编写一个解码程序。首先输入一个编码头（例如 AB#TANCnrtXc），则上述序列的每个串依次对应编码头的每个字符。例如，0 对应 A，00 对应 B，01 对应 #，...，110 对应 X，0000 对应 c。接下来是编码文本（可能由多行组成，你应当把它们拼成一个长长的 01 串）。编码文本由多个小节组成，每个小节的前 3 个数字代表小节中每个编码的长度（用二进制表示，例如 010 代表长度为 2），然后是各个字符的编码，以全 1 结束（例如，编码长度为 2 的小节以 11 结束）。编码文本以编码长度为 000 的小节结束。

例如，编码头为 \$**\，编码文本为 0100000101101100011100101000，应这样解码：010(编码长度为 2)00(#)00(#)10(*)11(小节结束)011(编码长度为 3)000(\)111(小节结束)001(编码长度为 1)0(\$)1(小节结束)000(编码结束)。

【分析】

还记得二进制吗？如果不记得，请重新翻阅第 3 章的最后部分。有了二进制，就不必以字符串的形式保存这一大串编码了，只需把编码理解成二进制，用(len, value)这个二元组来表示一个编码，其中 len 是编码长度，value 是编码对应的十进制值。如果用 codes[len][value] 保存这个编码所对应的字符，则主程序看上去应该是这个样子的。

```
#include<stdio.h>
#include<string.h> //使用 memset
int readchar() { ... }
int readint(int c) { ... }
```

```
int code[8][1<<8];
```

```
int readcodes() { ... }
```

```
int main() {
```

```
    while(readcodes()) { //无法读取更多编码头时退出
```

```
        //printcodes();
```

```
        for(;;) {
```

```

int len = readint(3);
if(len == 0) break;
//printf("len=%d\n", len);
for(;;) {
    int v = readint(len);
    //printf("v=%d\n", v);
    if(v == (1 << len)-1) break;
    putchar(code[len][v]);
}
putchar('\n');
}
return 0;
}

```

主程序里接连使用了两个还没有介绍的函数：`readcodes` 和 `readint`。前者用来读取编码，后者读取 `c` 位二进制字符（即 0 和 1），并转化为十进制整数。

本题的调试方法也很有代表性。上面的代码中已经包含了几条注释掉的 `printf` 语句，用于打印出一些关键变量的值。如果程序的输出不是想要的结果，题目中的举例就派上用场了：只需把举例中的解释和程序输出的中间结果一一对照，就能知道问题出在哪里。

编写 `readint` 时会遇到同一个问题：如何处理“编码文本可以由多行组成”这个问题？方法有很多种，笔者的方案是再编写一个“跨行读字符”的函数 `readchar`。

```

int readchar() {
    for(;;) {
        int ch = getchar();
        if(ch != '\n' && ch != '\r') return ch; //一直读到非换行符为止
    }
}

int readint(int c) {
    int v = 0;
    while(c--) v = v * 2 + readchar() - '0';
    return v;
}

```

下面是函数 `readcodes`。首先使用 `memset` 清空数组（这是个好习惯。还记得之前讲过的多数据题目的常见错误吗？），编码头自身占一行，所以应该用 `readchar` 读取第一个字符，而用普通的 `getchar` 读取剩下的字符，直到 `\n`。这样做，代码比较简单，但有些读者可能会觉得有些别扭。没关系，你完全可以使用另外一套自己觉得更清晰的方法。

```

int readcodes() {
    memset(code, 0, sizeof(code)); //清空数组
}

```



```

code[1][0] = readchar(); //直接调到下一行开始读取。如果输入已经结束，会读到 EOF
for(int len = 2; len <= 7; len++) {
    for(int i = 0; i < (1<<len)-1; i++) {
        int ch = getchar();
        if(ch == EOF) return 0;
        if(ch == '\n' || ch == '\r') return 1;
        code[len][i] = ch;
    }
}
return 1;
}

```

最后是前面提到的 `printcodes` 函数。这个函数对于解题来说不是必需的，但对于调试却是有用的。

```

void printcodes() {
    for(int len = 1; len <= 7; len++)
        for(int i = 0; i < (1<<len)-1; i++) {
            if(code[len][i] == 0) return;
            printf("code[%d][%d] = %c\n", len, i, code[len][i]);
        }
}

```

由于每次读取编码头时把 `codes` 数组清空了，所以只要遇到字符为 0 的情况，就表示编码头已经结束。

例题 4-5 追踪电子表格中的单元格 (Spreadsheet Tracking, ACM/ICPC World Finals 1997, UVa512)

有一个 r 行 c 列 ($1 \leq r, c \leq 50$) 的电子表格，行从上到下编号为 $1 \sim r$ ，列从左到右编号为 $1 \sim c$ 。如图 4-2 (a) 所示，如果先删除第 1、5 行，然后删除第 3, 6, 7, 9 列，结果如图 4-2 (b) 所示。

	A	B	C	D	E	F	G	H	I
1	22	55	66	77	88	99	10	12	14
2	2	24	6	8	22	12	14	16	18
3	18	19	20	21	22	23	24	25	26
4	24	25	26	67	22	69	70	71	77
5	68	78	79	80	22	25	28	29	30
6	16	12	11	10	22	56	57	58	59
7	33	34	35	36	22	38	39	40	41

(a)

	A	B	C	D	E
1	2	24	8	22	16
2	18	19	21	22	25
3	24	25	67	22	71
4	16	12	10	22	58
5	33	34	36	22	40

(b)

图 4-2 删除行、列

接下来在第 2、3、5 行前各插入一个空行，然后在第 3 列前插入一个空列，会得到如图 4-3 所示结果。

	A	B	C	D	E	F
1	2	24		8	22	16
2						
3	18	19		21	22	25
4						
5	24	25		67	22	71
6	16	12		10	22	58
7						
8	33	34		36	22	40

图 4-3 插入行、列

你的任务是模拟这样的 n 个操作。具体来说一共有 5 种操作:

- EX $r_1\ c_1\ r_2\ c_2$ 交换单元格(r_1, c_1), (r_2, c_2)。
- <command> A $x_1\ x_2\ \dots\ x_A$ 插入或删除 A 行或列 (DC-删除列, DR-删除行, IC-插入列, IR-插入行, $1 \leq A \leq 10$)。

在插入/删除指令后, 各个 x 值不同, 且顺序任意。接下来是 q 个查询, 每个查询格式为 “rc”, 表示查询原始表格的单元格(r, c)。对于每个查询, 输出操作执行完后该单元格的新位置。输入保证在任意时刻行列数均不超过 50。

【分析】

最直接的思路就是首先模拟操作, 算出最后的电子表格, 然后在每次查询时直接在电子表格中找到所求的单元格。为了锻炼读者的代码阅读能力, 此处不对代码进行任何解释:

```
#include<stdio.h>
#include<string.h>
#define maxd 100
#define BIG 10000
int r, c, n, d[maxd][maxd], d2[maxd][maxd], ans[maxd][maxd], cols[maxd];

void copy(char type, int p, int q) {
    if(type == 'R') {
        for(int i = 1; i <= c; i++)
            d[p][i] = d2[q][i];
    } else {
        for(int i = 1; i <= r; i++)
            d[i][p] = d2[i][q];
    }
}

void del(char type) {
    memcpy(d2, d, sizeof(d));
    int cnt = type == 'R' ? r : c, cnt2 = 0;
    for(int i = 1; i <= cnt; i++) {
        if(!cols[i]) copy(type, ++cnt2, i);
    }
}
```

```

    if(type == 'R') r = cnt2; else c = cnt2;
}

void ins(char type) {
    memcpy(d2, d, sizeof(d));
    int cnt = type == 'R' ? r : c, cnt2 = 0;
    for(int i = 1; i <= cnt; i++) {
        if(cols[i]) copy(type, ++cnt2, 0);
        copy(type, ++cnt2, i);
    }
    if(type == 'R') r = cnt2; else c = cnt2;
}

int main() {
    int r1, c1, r2, c2, q, kase = 0;
    char cmd[10];
    memset(d, 0, sizeof(d));
    while(scanf("%d%d%d", &r, &c, &n) == 3 && r) {
        int r0 = r, c0 = c;
        for(int i = 1; i <= r; i++)
            for(int j = 1; j <= c; j++)
                d[i][j] = i*BIG + j;
        while(n--) {
            scanf("%s", cmd);
            if(cmd[0] == 'E') {
                scanf("%d%d%d%d", &r1, &c1, &r2, &c2);
                int t = d[r1][c1]; d[r1][c1] = d[r2][c2]; d[r2][c2] = t;
            } else {
                int a, x;
                scanf("%d", &a);
                memset(cols, 0, sizeof(cols));
                for(int i = 0; i < a; i++) { scanf("%d", &x); cols[x] = 1; }
                if(cmd[0] == 'D') del(cmd[1]); else ins(cmd[1]);
            }
        }
        memset(ans, 0, sizeof(ans));
        for(int i = 1; i <= r; i++)
            for(int j = 1; j <= c; j++) {
                ans[d[i][j]/BIG][d[i][j]%BIG] = i*BIG+j;
            }
        if(kase > 0) printf("\n");
        printf("Spreadsheet #%d\n", ++kase);
    }
}

```

```
scanf("%d", &q);
while(q--) {
    scanf("%d%d", &r1, &c1);
    printf("Cell data in (%d,%d) ", r1, c1);
    if(ans[r1][c1] == 0) printf("GONE\n");
    else printf("moved to (%d,%d)\n", ans[r1][c1]/BIG, ans[r1][c1]%BIG);
}
}
return 0;
}
```

另一个思路是将所有操作保存, 然后对于每个查询重新执行每个操作, 但不需要计算整个电子表格的变化, 而只需关注所查询的单元格的位置变化。对于题目给定的规模来说, 这个方法不仅更好写, 而且效率更高。代码如下:

```
#include<stdio.h>
#include<string.h>
#define maxd 10000

struct Command {
    char c[5];
    int r1, c1, r2, c2;
    int a, x[20];
} cmd[maxd];

int r, c, n;

int simulate(int* r0, int* c0) {
    for(int i = 0; i < n; i++) {
        if(cmd[i].c[0] == 'E') {
            if(cmd[i].r1 == *r0 && cmd[i].c1 == *c0) { *r0 = cmd[i].r2; *c0 = cmd[i].c2; }
            else if(cmd[i].r2 == *r0 && cmd[i].c2 == *c0) { *r0 = cmd[i].r1; *c0 = cmd[i].c1; }
        } else {
            int dr = 0, dc = 0;
            for(int j = 0; j < cmd[i].a; j++) {
                int x = cmd[i].x[j];
                if(cmd[i].c[0] == 'I') {
                    if(cmd[i].c[1] == 'R' && x <= *r0) dr++;
                    if(cmd[i].c[1] == 'C' && x <= *c0) dc++;
                }
            }
            else {
                if(cmd[i].c[1] == 'R' && x == *r0) return 0;
            }
        }
    }
}
```

```

        if(cmd[i].c[1] == 'C' && x == *c0) return 0;
        if(cmd[i].c[1] == 'R' && x < *r0) dr--;
        if(cmd[i].c[1] == 'C' && x < *c0) dc--;
    }
}
*r0 += dr; *c0 += dc;
}
return 1;
}

int main() {
    int r0, c0, q, kase = 0;
    while(scanf("%d%d%d", &r, &c, &n) == 3 && r) {
        for(int i = 0; i < n; i++) {
            scanf("%s", cmd[i].c);
            if(cmd[i].c[0] == 'E') {
                scanf("%d%d%d%d", &cmd[i].r1, &cmd[i].c1, &cmd[i].r2, &cmd[i].c2);
            } else {
                scanf("%d", &cmd[i].a);
                for(int j = 0; j < cmd[i].a; j++) scanf("%d", &cmd[i].x[j]);
            }
        }
        if(kase > 0) printf("\n");
        printf("Spreadsheet #%d\n", ++kase);

        scanf("%d", &q);
        while(q--) {
            scanf("%d%d", &r0, &c0);
            printf("Cell data in (%d,%d) ", r0, c0);
            if(!simulate(&r0, &c0)) printf("GONE\n");
            else printf("moved to (%d,%d)\n", r0, c0);
        }
    }
    return 0;
}

```

有没有觉得 `simulate` 函数不是特别自然？因为所有用到 `r0` 和 `c0` 的地方都要加上一个星号。幸运的是，C++语言中有另外一个语法，可以更自然地表达这种“需要被修改的参数”，详见第5章中的“引用”部分。

例题 4-6 师兄帮帮忙 (A Typical Homework (a.k.a Shi Xiong Bang Bang Mang), Rujia Liu's Present 5, UVa 12412)

(题目背景略，有兴趣的读者请自行阅读原题)

编写一个成绩管理系统（SPMS）。最多有 100 个学生，每个学生有如下属性。

- ☐ SID: 学生编号，包含 10 位数字。
- ☐ CID: 班级编号，为不超过 20 的正整数。
- ☐ 姓名: 不超过 10 的字母和数字组成，第一个字符为大写字母。名字中不能有空白字符。
- ☐ 4 门课程（语文、数学、英语、编程）成绩，均为不超过 100 的非负整数。

进入 SPMS 后，应显示主菜单：

```
Welcome to Student Performance Management System (SPMS).
```

```
1 - Add
2 - Remove
3 - Query
4 - Show ranking
5 - Show Statistics
0 - Exit
```

选择 1 之后，会出现添加学生记录的提示信息：

```
Please enter the SID, CID, name and four scores. Enter 0 to finish.
```

然后等待输入。本题保证输入总是合法的（不会有非法的 SID、CID，并且恰好有 4 个分数等），但可能会输入重复 SID。在这种情况下，需要输出一行提示：

```
Duplicated SID.
```

不过名字是可以重复的。你的程序应当不停地打印前述提示信息，直到用户输入单个 0。然后应当再次打印主菜单。

选择 2 之后，会出现如下提示信息：

```
Please enter SID or name. Enter 0 to finish.
```

然后等待输入，在数据库中删除能匹配上述 SID 或者名字的所有学生，并且打印如下信息（xx 可以等于 0）：

```
xx student(s) removed.
```

你的程序应当不停地打印前述提示信息，直到用户输入单个 0，然后再次打印主菜单。

选择 3 之后，会出现如下提示信息：

```
Please enter SID or name. Enter 0 to finish.
```

然后等待输入。如果数据库中没有能匹配上述 SID 或者名字的学生，什么都不要做；否则输出所有满足条件的学生，按照进入数据库的顺序排列。输出格式和添加的格式相同，但增加 3 列：年级排名（第一列）、总分和平均分（最后两列）。所有班级中总分最高的学生获得第 1 名，如果有两个学生并列第 2 名，则下一个学生的排名为 4（而非 3）。你的

程序应当不停地打印前述提示信息，直到用户输入单个 0。然后应当再次打印主菜单。

选择 4 之后，会出现如下提示信息：

```
Showing the ranklist hurts students' self-esteem. Don't do that.
```

然后自动返回主菜单。

选择 5 之后，会出现如下提示信息：

```
Please enter class ID, 0 for the whole statistics.
```

当用户输入班级 ID 之后（0 代表全年级），显示如下信息（注意，“及格”是指分数不小于 60 分）：

```
Chinese
```

```
Average Score: xx.xx
```

```
Number of passed students: xx
```

```
Number of failed students: xx
```

...（为了节约篇幅，此处省略了 Mathematics、English 和 Programming 的统计信息）

```
Overall:
```

```
Number of students who passed all subjects: xx
```

```
Number of students who passed 3 or more subjects: xx
```

```
Number of students who passed 2 or more subjects: xx
```

```
Number of students who passed 1 or more subjects: xx
```

```
Number of students who failed all subjects: xx
```

然后自动回到主菜单。

选择 0 之后，程序终止。注意，单科成绩和总分都应格式化为整数，但平均分应恰好保留两位小数。

提示：这个程序适合直接运行，用键盘与之交互，然后从屏幕中看到输出信息。但正因为如此，作为一道算法竞赛的题目，其输出看上去会比较乱。

【分析】

正如题目所说，这是一道很常见的“作业题”，在一些早期的大学编程教材中可以看到类似的问题（只是要求不一定有这么明确）。

因为要求比较多，可以沿用之前介绍过的“自顶向下，逐步求精”方法，先写出如下框架：

```
int main() {
    for(;;) {
        int choice;
        print_menu();
        scanf("%d", &choice);
        if(choice == 0) break;
```

```

    if(choice == 1) add();
    if(choice == 2) DQ(0);
    if(choice == 3) DQ(1);
    if(choice == 4) printf("Showing the ranklist hurts students' self-esteem.
Don't do that.\n");
    if(choice == 5) stat();
}
return 0;
}

```

接下来就是分别实现各个函数了。注意上面把操作 2 (删除) 和操作 3 (查询) 合并在了一起, 因为二者非常相似, 代码如下 (isq=1 表示查询, isq=0 表示删除):

```

void DQ(int isq) {
    char s[maxl];
    for(;;) {
        printf("Please enter SID or name. Enter 0 to finish.\n");
        scanf("%s", s);
        if(strcmp(s, "0") == 0) break;
        int r = 0;
        for(int i = 0; i < n; i++) if(!removed[i]) {
            if(strcmp(sid[i], s) == 0 || strcmp(name[i], s) == 0) {
                if(isq) printf("%d %s %d %s %d %d %d %d %.2f\n", rank(i), sid[i],
cid[i], name[i], score[i][0], score[i][1], score[i][2], score[i][3], score[i][4],
score[i][4]/4.0+EPS);
                else { removed[i] = 1; r++; }
            }
        }
        if(!isq) printf("%d student(s) removed.\n", r);
    }
}

```

在编写上述函数的过程中, 用到了尚未编写的 rank 函数, 并且直接使用了还没有声明的数组 removed、sid、cid、name 和 score。换句话说, 根据函数编写的需要定义了数据结构, 而不是一开始就设计好数据结构。程序的其他部分略为麻烦, 但没有难点, 建议初学者自主完成整个程序, 作为 C 语言部分的结束。

顺便说一句, 虽然在前面学习了排序, 但 rank 函数的实现并不一定要对数据排序。另外, 上述代码在输出实数时加了一个 EPS, 原因将在本章最后讨论。

4.5 注解与习题

到目前为止, 本书要介绍的 C 语言知识已经全部讲完了 (第 5 章将介绍 C++)。本章

涉及了整个 C 语言中最难理解的两项内容：指针和递归。

4.5.1 头文件、副作用及其他

还记得第1章中给出的程序框架吗？是时候搞清楚所有细节了。读者现在已经知道 `main` 函数也是一个普通的函数（甚至可以递归调用），其返回值将告之操作系统，在算法竞赛中应当总是等于 0，唯一的谜团就是 `#include<stdio.h>` 了。

这是一个头文件。什么是头文件呢？实践者的理解方式就是——不加这一行时会出现什么错误，反过来就说明了这一行的作用。不加这一行的编译警告是：

```
warning: incompatible implicit declaration of built-in function 'printf'
[enabled by default]
```

也就是说，`printf` 函数的“隐式定义”出了问题，这个头文件和 `printf` 有关。还记得第一次介绍 `math.h` 是怎么讲的吗？如果要使用数学相关的函数，需要包含这个头文件。换句话说，头文件的作用就是：包含了一些函数，供主程序使用^①。表 4-1 中列出了一些常用函数和对应的头文件。

表 4-1 常用函数及头文件

函 数	作 用	头 文 件
<code>printf/scanf</code> 及其“兄弟”	格式化输入输出	<code>stdio.h</code>
<code>fopen, freopen, fclose</code>	文件的打开与关闭	
<code>getchar, fgetc</code> 等	字符/字符串输入输出	
<code>sin/cos/pow</code> 等	各种数学函数	<code>math.h</code>
<code>strlen, strcat</code>	字符串函数	<code>string.h</code>
<code>memset, memcpy</code>	内存清 0 与赋值	
<code>isalpha, isdigit, toupper</code> 等	字符分类与转换	<code>ctype.h</code>
<code>clock</code>	计时函数	<code>time.h</code>

在编写实用软件时，往往需要编写自己的头文件，但在大部分算法竞赛中，只是编写单个程序文件。在本书中，所有题目都由单个程序文件求解。

下面来看一个有意思的问题：是否可以编写一个函数 `f()`，使得依次执行 `int a = f()` 和 `int b = f()` 以后 `a` 和 `b` 的值不同？使用全局变量，这个问题不难解决：

```
#include<stdio.h>

int g = 0;

int f() { g++; return g; } //修改全局变量的函数

int main() {
    int a = f();
```

^① 和本章开头的自定义函数不同，头文件里并没有 `printf` 的源代码，而只有它的声明。`printf` 属于 `libc` 的一部分，有兴趣的读者请自行查阅相关资料。

```
int b = f();
printf("%d %d\n", a, b);
return 0;
}
```

不难写出一个更有意思的程序：写3个函数 $f()$ 、 $g()$ 和 $h()$ ，使得 “ $\text{int } a = (f() + g()) + h()$ ” 和 “ $\text{int } b = f() + (g() + h())$ ” 后， a 和 b 的值不同。

加法明明满足结合律，居然有可能 “ $(f() + g()) + h()$ ” 不等于 “ $f() + (g() + h())$ ”！这个例子说明：C 语言的函数并不都像数学函数那样“规矩”。或者说得学术一点：C 语言的函数可以有副作用，而不像数学函数那样“纯”。本书无意深入介绍函数式编程，但时刻警惕并最小化“副作用”是一个良好的编程习惯。正因为如此，前面曾多次强调：全局变量要少用。

再来看一个小问题：函数可以返回指针吗？例如这样：

```
int* get_pointer() {
    int a = 3;
    return &a;
}
```

这个程序可以编译通过，不过有一个警告：

```
warning: function returns address of local variable [enabled by default]
```

意思是函数返回了一个局部变量的地址。为什么不能返回局部变量的地址呢？前面说过，局部变量是在栈中，函数执行完毕后，局部变量就失效了。严格地讲，指针里保存的地址仍然存在，但不再属于那个局部变量了。这时如果修改那个指针指向的内容，程序有可能会崩溃，也可能悄悄地修改了另外一个变量的值，使程序输出一个莫名其妙的结果。

那推荐的写法是怎样的？这取决于你想做什么。如果只是想得到一个指向内容为3的指针，可以把这个指针作为参数，然后在函数里修改它；如果坚持返回一个“新”的指针，可以使用 `malloc` 函数进行动态内存分配。笔者并不准备在这里叙述详细做法，因为在接下来的章节中会对动态内存分配进行深入讨论。在学习到那些知识之前，请尽量不要编写返回指针的函数。

最后一个话题是关于浮点误差的。例如：

```
#include<stdio.h>

int main() {
    double f;
    for(f = 2; f > 1; f -= 1e-6);
    printf("%.7f\n", f);
    printf("%.7f\n", f / 4);
    printf("%.1f\n", f / 4);
    return 0;
}
```

在笔者的机器上，输出如下：

```
0.9999990
0.2499998
0.2
```

换句话说，在不断减 $1e-6$ 的过程中出现了误差，使得循环终止时 f 并不等于 1，而是比 1 小一点。在除以 4 保留 1 位小数时成了 0.2。如果不出现误差，正确答案应该是 0.25 四舍五入保留一位小数，即 0.3。一道好的竞赛题目应避免这种情况出现^①，但作为参赛选手来说，有一种方法可以缓解这种情况：加上一个 EPS 以后再输出。这里的 EPS 通常取一个比最低精度还要小几个 3 个数量级的小实数。例如，要求保留 3 位小数时取 EPS 为 $1e-6$ 。这只是个权宜之计，甚至有可能起到“反作用”（如正确答案真的是 0.499999），但在实践中很好用（毕竟正确答案是 0.499999 的情况比 0.5 要少很多）。

4.5.2 例题一览和习题

本章共有 6 道例题，如表 4-2 所示。除了最后两道题目比较复杂之外，读者应熟练掌握前 4 道题目的程序写法。当然，为了巩固基础，让后面的学习更加轻松，笔者强烈建议大家独立实现所有 6 道题目。

表 4-2 例题一览

类 别	题 号	题目名称（英文）	备 注
例题 4-1	UVa1339	Ancient Cipher	排序
例题 4-2	UVa489	Hangman Judge	自顶向下逐步求精法
例题 4-3	UVa133	The Dole Queue	子过程（函数）设计
例题 4-4	UVa213	Message Decoding	二进制；输入技巧；调试技巧
例题 4-5	UVa512	Spreadsheet Tracking	模拟；一题多解
例题 4-6	UVa12412	A Typical Homework (a.k.a Shi Xiong Bang Bang Mang)	综合练习

下面是一些习题。这些题目的综合性较强，部分题目还涉及一些专门知识（如中国象棋、莫尔斯电码、RAID），理解起来也需要一定时间。另外一些题目需要一些思考，否则无从入手编写程序。由于这些题目的挑战性，在继续阅读之前只需完成其中的 3 道题目。如果想达到更好的效果，最好是完成 3 道或更多的题目。

习题 4-1 象棋 (Xiangqi, ACM/ICPC Fuzhou 2011, UVa1589)

考虑一个象棋残局，其中红方有 n ($2 \leq n \leq 7$) 个棋子，黑方只有一个将。红方除了有一个帅 (G) 之外还有 3 种可能的棋子：车 (R)，马 (H)，炮 (C)，并且需要考虑“蹩马腿”（如图 4-4 所示）与将和帅不能照面（将、帅如果同在一条直线上，中间又不隔着任何棋子的情况下，走子的一方获胜）的规则。

输入所有棋子的位置，保证局面合法并且红方已经将军。你的任务是判断红方是否已

^① 方法有两种：一是删除答案恰好处于“舍入交界口”的数据，二是允许选手输出和标准答案有少许出入。

经把黑方将死。关于中国象棋的相关规则请参见原题。

习题 4-2 正方形 (Squares, ACM/ICPC World Finals 1990, UVa201)

有 n 行 n 列 ($2 \leq n \leq 9$) 的小黑点，还有 m 条线段连接其中的一些黑点。统计这些线段连成了多少个正方形（每种边长分别统计）。

行从上到下编号为 $1 \sim n$ ，列从左到右编号为 $1 \sim n$ 。边用 H_{ij} 和 V_{ij} 表示，分别代表边 $(i,j)-(i,j+1)$ 和 $(i,j)-(i+1,j)$ 。如图 4-5 所示最左边的线段用 V_{11} 表示。图中包含两个边长为 1 的正方形和一个边长为 2 的正方形。

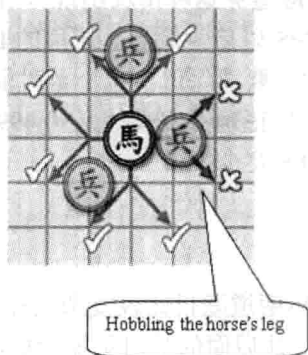


图 4-4 “蹩马腿”情况

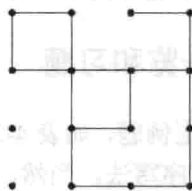


图 4-5 正方形

习题 4-3 黑白棋 (Othello, ACM/ICPC World Finals 1992, UVa220)

你的任务是模拟黑白棋游戏的进程。黑白棋的规则为：黑白双方轮流放棋子，每次必须让新放的棋子“夹住”至少一枚对方棋子，然后把所有被新放棋子“夹住”的对方棋子替换成己方棋子。一段连续（横、竖或者斜向）的同色棋子被“夹住”的条件是两端都是对方棋子（不能是空位）。如图 4-6 (a) 所示，白棋有 6 个合法操作，分别为 (2,3), (3,3), (3,5), (6,2), (7,3), (7,4)。选择在 (7,3) 放白棋后变成如图 4-6 (b) 所示效果（注意有竖向和斜向的共两枚黑棋变白）。注意 (4,6) 的黑色棋子虽然被夹住，但不是被新放的棋子夹住，因此不变白。

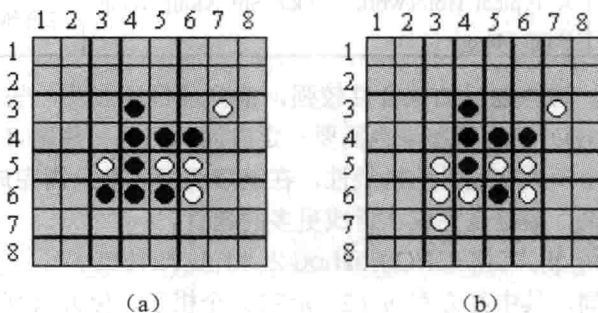


图 4-6 黑白棋

输入一个 8×8 的棋盘以及当前下一次操作的游戏者，处理 3 种指令：

- L 指令打印所有合法操作，按照从上到下，从左到右的顺序排列（没有合法操作时输出 No legal move）。
- Mrc 指令放一枚棋子在 (r,c) 。如果当前游戏者没有合法操作，则是先切换游戏者再

操作。输入保证这个操作是合法的。输出操作完毕后黑白方的棋子总数。

□ Q 指令退出游戏，并打印当前棋盘（格式同输入）。

习题 4-4 骰子涂色 (Cube painting, UVa 253)

输入两个骰子，判断二者是否等价。每个骰子用 6 个字母表示，如图 4-7 所示。

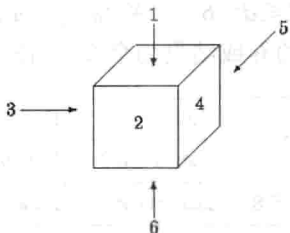


图 4-7 骰子涂色

例如 `rbgggr` 和 `rggbgr` 分别表示如图 4-8 所示的两个骰子。二者是等价的，因为图 4-8 (a) 所示的骰子沿着竖直轴旋转 90° 之后就可以得到图 4-8 (b) 所示的骰子。

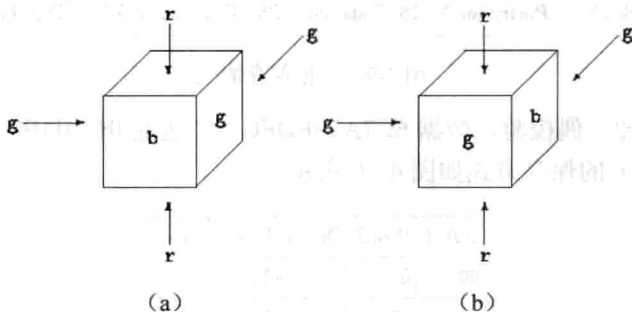


图 4-8 旋转前后的两个骰子

习题 4-5 IP 网络 (IP Networks, ACM/ICPC NEERC 2005, UVa1590)

可以用一个网络地址和一个子网掩码描述一个子网（即连续的 IP 地址范围）。其中子网掩码包含 32 个二进制位，前 $32-n$ 位为 1，后 n 位为 0，网络地址的前 $32-n$ 位任意，后 n 位为 0。所有前 $32-n$ 位和网络地址相同的 IP 都属于此网络。

例如，网络地址为 194.85.160.176（二进制为 11000010|01010101|10100000|10110000），子网掩码为 255.255.255.248（二进制为 11111111|11111111|11111111|11111000），则该子网的 IP 地址范围是 194.85.160.176~194.85.160.183。输入一些 IP 地址，求最小的网络（即包含 IP 地址最少的网络），包含所有这些输入地址。

例如，若输入 3 个 IP 地址：194.85.160.177、194.85.160.183 和 194.85.160.178，包含上述 3 个地址的最小网络的网络地址为 194.85.160.176，子网掩码为 255.255.255.248。

习题 4-6 莫尔斯电码 (Morse Mismatches, ACM/ICPC World Finals 1997, UVa508)

输入每个字母的 Morse 编码，一个词典以及若干个编码。对于每个编码，判断它可能是哪个单词。如果有多个单词精确匹配，任选一个输出并且后面加上“!”；如果无法精确匹配，可以在编码尾部增加或删除一些字符以后匹配某个单词（增加或删除的字符应尽量少）。如果有多个单词可以这样匹配上，任选一个输出并且在后面加上“?”。

莫尔斯电码的细节参见原题。

习题 4-7 RAID 技术 (RAID!, ACM/ICPC World Finals 1997, UVa509)

RAID 技术用多个磁盘保存数据。每份数据在不只一个磁盘上保存, 因此在某个磁盘损坏时能通过其他磁盘恢复数据。本题讨论其中一种 RAID 技术。数据被划分成大小为 s ($1 \leq s \leq 64$) 比特的数据块保存在 d ($2 \leq d \leq 6$) 个磁盘上, 如图 4-9 所示, 每 $d-1$ 个数据块都有一个校验块, 使得每 d 个数据块的异或结果为全 0 (偶校验) 或者全 1 (奇校验)。

Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
Parity for 1-4	Data block 1	Data block 2	Data block 3	Data block 4
Data block 5	Parity for 5-8	Data block 6	Data block 7	Data block 8
Data block 9	Data block 10	Parity for 9-12	Data block 11	Data block 12
Data block 13	Data block 14	Data block 15	Parity for 13-16	Data block 16
Data block 17	Data block 18	Data block 19	Data block 20	Parity for 17-20
Parity for 21-24	Data block 21	Data block 22	Data block 23	Data block 24
Data block 25	Parity for 25-28	Data block 26	Data block 27	Data block 28

图 4-9 数据保存情况

例如, $d=5$, $s=2$, 偶校验, 数据 6C7A79EDFC (二进制 01101100 01111010 01111001 11101101 11111100) 的保存方式如图 4-10 所示。

Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
00	01	10	11	00
01	10	11	10	10
01	11	01	10	01
11	10	11	11	01
11	11	11	00	11

图 4-10 数据 6C7A79EDPC 的保存方式

其中加粗块是校验块。输入 d 、 s 、 b 、校验的种类 (E 表示偶校验, O 表示奇校验) 以及 b ($1 \leq b \leq 100$) 个数据块 (其中 “?” 表示损坏的数据), 你的任务是恢复并输出完整的数据。如果校验错或者由于损坏数据过多无法恢复, 应报告磁盘非法。

提示: 本题是位运算的不错练习, 但如果没有 RAID 的知识背景, 上述简要翻译可能较难理解, 细节建议参考原题。

习题 4-8 特别困的学生 (Extraordinarily Tired Students, ACM/ICPC Xi'an 2006, UVa12108)

课堂上有 n 个学生 ($n \leq 10$)。每个学生都有一个“睡眠-清醒”周期, 其中第 i 个学生醒 A_i 分钟后睡 B_i 分钟, 然后重复 ($1 \leq A_i, B_i \leq 5$), 初始时第 i 个学生处在他的周期的第 C_i 分钟。每个学生在临睡前会察看全班睡觉人数是否严格大于清醒人数, 只有这个条件满足时才睡觉, 否则就坚持听课 A_i 分钟后再次检查这个条件。问经过多长时间后全班都清醒。

如果用(A,B,C)描述一些学生,则图 4-11 中描述了 3 个学生(2,4,1)、(1,5,2)和(1,4,3)在每个时刻的行为。

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
		☹	☹	☹	☹			☹	☹	☹	☹						
☹	☹	☹	☹	☹		☹	☹	☹	☹	☹		☹	☹	☹	☹	☹	
☹	☹	☹		☹	☹	☹	☹		☹	☹	☹						

图 4-11 3 个学生每个时刻的行为

注意:有可能并不存在“全部都清醒”的时刻,此时应输出-1。

习题 4-9 数据挖掘 (Data Mining, ACM/ICPC NEERC 2003, UVa1591)

有两个 n 元素数组 P 和 Q 。 P 数组每个元素占 S_P 个字节, Q 数组每个元素占 S_Q 个字节。有时需直接根据 P 数组中某个元素 $P(i)$ 的偏移量 $P_{\text{ofs}}(i)$ 算出对应的 $Q(i)$ 的偏移量 $Q_{\text{ofs}}(i)$ 。当两个数组的元素均为连续存储时 $Q_{\text{ofs}}(i) = P_{\text{ofs}}(i) / S_P * S_Q$, 但因为除法慢, 可以把式子改写成速度较快的 $Q_{\text{ofs}}(i) = (P_{\text{ofs}}(i) + P_{\text{ofs}}(i) << A) >> B$ 。为了让这个式子成立, 在 P 数组仍然连续存储的前提下, Q 数组可以不连续存储(但不同数组元素的存储空间不能重叠)。这样做虽然会浪费一些空间, 但是提升了速度, 是一种用空间换时间的方法。

输入 n 、 S_P 和 S_Q ($N \leq 2^{20}$, $1 \leq S_P, S_Q \leq 2^{10}$), 你的任务是找到最优的 A 和 B , 使得占的空间 K 尽量小。输出 K 、 A 、 B 的值。多解时让 A 尽量小, 如果仍多解则让 B 尽量小。

提示: 本题有一定实际意义, 不过描述比较抽象。如果对本题兴趣不大, 可以先跳过。

习题 4-10 洪水! (Flooded! ACM/ICPC World Finals 1999, UVa815)

有一个 $n*m$ ($1 \leq m, n < 30$) 的网格, 每个格子是边长 10 米的正方形, 网格四周是无限大的墙壁。输入每个格子的海拔高度, 以及网格内雨水的总体积, 输出水位的海拔高度以及有多少百分比的区域有水(即高度严格小于水平面)。

本题有多种方法, 能锻炼思维, 建议读者一试。

4.5.3 小结

指针还有很多相关内容本书没有介绍, 例如, 指向 `void` 型的指针、指向函数的指针、指向常量的指针以及指针和数组之间的关系(注意, 尽管在很多地方可以混用, 但指针和数组不是一回事!《C 语言程序设计奥秘》用一章的篇幅来叙述二者的区别)。正如书中所说, 本书将尽量回避指针, 但尽管如此, 调试并理解前面几个 `swap` 函数的工作方式对于理解计算机的工作原理大有好处。

递归需要从概念和语言两个方面理解。从概念上, 递归就是“自己使用自己”的意思。递归调用就是自己调用自己, 递归定义就是自己定义自己……当然, 这里的“使用自己”可以是直接的, 也可以是间接的。很多初学者在学习递归时专注于表象, 从而未能透彻理解其“计算机”本质。由于我们的重点是设计算法和编写程序, 理解递归函数的执行过程是非常重要的。因此, 本章大量使用了 `gdb` 作为工具讲解内部机理, 即使读者在平时编程时不用 `gdb` 调试, 在学习初期用它帮助理解也是大有裨益的。关于 `gdb` 的更多介绍参见附录 A。